



SYSTEM DESIGN

Caching in AI

Systems

Faster · Cheaper · Scalable

Caching

AI Systems

Memory



Chandra Sekhar

AI Engineer & Trainer

What Is Caching?

Store the result of an expensive operation once,
then reuse it every time after.

THE SIMPLE IDEA

Do it once → reuse many times

First Time

User asks "What is machine learning?" → LLM generates
answer → **stored**

Next Time

Same question asked → **return stored response instantly** —
zero LLM call



Why It Matters

LLM calls are slow, APIs charge per token, and the same queries hit your system over and over.

✗ Without Cache

Same query → repeated LLM calls. High cost and high latency every time.

✓ With Cache

Same query → instant response. Zero compute cost on repeat.

EXAMPLE SCENARIO

100 users ask the same question:

✗ Without cache: 100 API calls

✓ With cache: 1 API call



Where It Fits

You can cache at multiple levels in the AI pipeline — not just one.



Input Layer

→ Prompt cache



Retrieval Layer

→ Document cache



Model Layer

→ KV cache



Output Layer

→ Response cache





Prompt Cache

Exact Input → Stored Output

Match identical prompts and serve the cached response — skipping the model entirely.

HOW IT WORKS

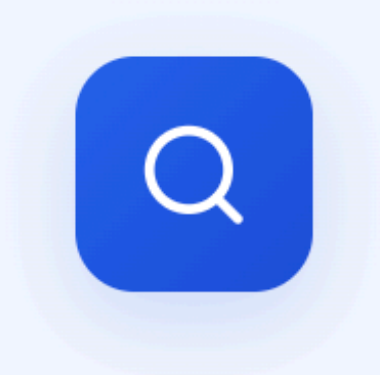
Key: Prompt text

Value: LLM response

Best for:

FAQs, repeated queries, and static knowledge requests with predictable input patterns.





Semantic Cache

Similar Meaning → Same Answer

Uses embeddings to detect meaning, not exact text — handles paraphrased queries automatically.

Query A

"What is AI?"

Query B

"Explain artificial intelligence"

→ Different text, same meaning → return cached answer

- 1 Convert query → embedding
- 2 Compare cosine similarity
- 3 If close enough → reuse answer



Retrieval Cache

Skip the Vector Search

In RAG pipelines, vector search is expensive. Cache the retrieved top-k documents for a semantic query instead of re-running it.

THE STRATEGY

Cache the **Top-k documents** for a semantic query.

Initial Query

"What is a vector database?" → retrieved docs are stored

Next Time

Skip vector search → use cached docs → save 80–300ms per query



The KV Cache

Inside the Transformer

Each new token must attend to all past tokens — which is extremely compute-heavy. The KV cache eliminates this re-computation.

What It Stores

Key + Value vectors of all past tokens in the attention mechanism.

The Benefit

- ⚡ Faster token generation
- ↓ Lower compute cost

Generating a sentence reuses past computations instead of recalculating every token from scratch.



Common Tools

The infrastructure used to implement caching layers in production.



Redis

Fast in-memory store for exact prompt caching.



FAISS

Vector index for semantic similarity lookup.



Elasticsearch

Hybrid document and retrieval caching.



Python Dict

Zero-setup local key-value cache for prototyping.



IMPLEMENTATION

Key → Value

Every cache layer maps down to a single Key-Value lookup system.

Prompt Cache

Key: Text
Value: Response

Semantic Cache

Key: Embedding
Value: Documents

KV Cache

Key: Token History | Value: Cached Vectors

PRO TIPS

- ✓ Use hashing for consistent keys
- ✓ Keep keys deterministic across requests
- ✓ Namespace keys to avoid conflicts





Time to Live

(TTL)

Caches shouldn't live forever. Without expiry policies your AI will serve stale or outdated answers to users.

News Chatbot

Cache expires in 1 hour to ensure fresh facts.

Static Knowledge

Cache lasts days, weeks, or indefinitely.

 Set TTL based on how fast your data changes — not as an afterthought.





Eviction

Strategies

When cache memory fills up, old data must be removed to make room for new entries.

LRU

Least Recently Used

Removes items that haven't been accessed in the longest time.

LFU

Least Frequently Used

Removes items that are accessed the least often overall.

 If memory is full, evict the least-used queries first to maintain a high cache hit rate.



When NOT to Cache

Wrong caching equals wrong output. Skip caching when data is too volatile or too personal.



Volatile Data

Data changes constantly — e.g. real-time stock prices, live sports scores.



Personalized Responses

Outputs are highly unique per user session and cannot be shared safely.



Sensitive Information

Strictly private queries like banking data or medical records.



End-to-End Example

User asks: **"What is RAG?"** — here's the full system flow:

1 Check response cache (exact match)

2 If miss → generate embedding

3 Check retrieval cache (semantic)

4 If miss → fetch docs from Vector DB

5 Call LLM → store response in cache

⚡ Next time: Instant answer — no LLM call needed



ENJOYED THIS BREAKDOWN?

Share it with your team

One repost could save a colleague hours
spent re-learning caching from scratch.

Repost

Save

Comment

 **Follow for more AI**